Reviewer Pack — Matroid Spread Bounded by Monotone DNF Size

Entry 27f88a70aaf4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:40:53 UTC

Matroid Spread Bounded by Monotone DNF Size

Entry ID: `27f88a70aaf4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 06:40:53 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory (polymatroid spread) **Field B** (complexity object): Monotone DNF Size for k-CLIQUE

Statement:

> For any monotone DNF formula Φ computing the k-CLIQUE function on n variables, the spread of the associated polymatroid (defined by Φ's clauses) satisfies spread(Φ) = Ω(k^{1/4} log n), and the DNF size is $n^{\{\Omega(k}\{1/4\})\}$.

Rationale (proposer's reasoning):

> The spread of the polymatroid captures the variability in the rank function, reflecting the structural complexity of the DNF. A higher spread indicates a more intricate dependency structure, necessitating exponential growth in the DNF size to represent the k-CLIQUE function.

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e78e028b0b294d84`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.9s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(n):
                if j != i:
                    factor = Fraction(A[j][i], A[i][i])
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return [row[i:] for row in A]

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def rank(matrix):
        reduced_matrix = gaussian_elimination(matrix)
        return sum(1 for row in reduced_matrix if any(row))

    def is_monotone_dnf(dnf):
        n = len(dnf[0])
        for clause in dnf:
            if not all(x == 0 or x == 1 for x in clause):
                return False
        return True

    def generate_k_clique_dnf(n, k):
        clauses = []
```

```
        for i in range(1 << n):
            if bin(i).count('1') == k:
                clause = [1 if (i >> j) & 1 else 0 for j in range(n)]
                clauses.append(clause)
        return clauses

    def dnf_size(dnf):
        return len(dnf)

    def spread(dnf):
        n = len(dnf[0])
        rank_function = [rank([clause[:i] + clause[i+1:] for clause in dnf]) for i in range(n)]
        return max(rank_function) - min(rank_function)

    n = 20
    k = random.randint(5, 10)
    dnf = generate_k_clique_dnf(n, k)

    if not is_monotone_dnf(dnf):
        return {
            "metric_name": "spread",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    spread_value = spread(dnf)
    dnf_size_value = dnf_size(dnf)

    return {
        "metric_name": "spread",
        "metric_value": spread_value,
        "instances_tested": 1,
        "conjecture_holds": spread_value >= k**(1/4) * math.log(n) and dnf_size_value >= n**(k**(1
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    spread_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(spread_values)/len(spread_values)} std=0.0 support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(spread_values)/len(spread_values)} std={math.sqrt(sum(
    else:
```

```
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"spread does not meet the conjectured bound\" fi
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04ff47ab.py", line 99, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04ff47ab.py", line 82, in run_trial
    spread_value = spread(dnf)
                   ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04ff47ab.py", line 66, in spread
    rank_function = [rank([clause[:i] + clause[i+1:] for clause in dnf]) for i in range(n)]
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04ff47ab.py", line 43, in rank
    reduced_matrix = gaussian_elimination(matrix)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_04ff47ab.py", line 28, in gaussian_elim
    factor = Fraction(A[j][i], A[i][i])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with ZeroDivisionError, preventing evaluation of the conjecture's validity. | next: Fix the Gaussian elimination implementation to handle zero denominators and re-run tests with diverse seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	104441
2	preregistration	ollama_remote	qwen3:8b	0	0	19931
3	novelty	ollama_remote	qwen3:8b	0	0	16616

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	qwen3:8b	0	0	10468
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14753
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11098
7	judge	ollama_remote	qwen3:8b	0	0	13017

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 190324 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/27f88a70aaf4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/27f88a70aaf4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/27f88a70aaf4.tar.gz` (if generated)